Faculty of Engineering and Technology

Department of Electrical and Computer Engineering

Operating System-ENCS3390

Programming Task Two

CPU Scheduling System Combined with Deadlock detection and Recovery

Prepared by:

Ibrahim Salahat **1221701**

Sadeen Faqeeh **1222177**

Instructor: **Dr. Abdel Salam Sayyad**

Section: Four

Date: January 12, 2024

Abstract

The project entails designing a simulator for the CPU scheduling system, which incorporates deadlock detection and recovery schemes. The system will be single-core CPU with multiple resource types to simulate scenarios of processes in real life, contending over their use. Priority-based scheduling is considered along with round-robin for equitable sharing and serving critical tasks with priority. In case of any detection of deadlocks, resolution through process termination would ensure continuity of the system. The Gantt chart measures performance metrics such as waiting time, turnaround time, and execution timeline. This simulation will provide proper management of resources along with a crucial scheduling technique for any modern operating system.

Contents

Abstract ---------- II

Table of Figures ---------- IV

List of Tables ---------- V

Introduction ---------- 1

Test Cases ---------- 2

- **Test Case 1: simple CPU Burst without IO** ---------- 2
- **Test Case 2: CPU Burst with IO and Resource** ---------- 3
- **Test Case 3: CPU Burst with IO and Resources** ---------- 4
- **Test Case 4: CPU Burst with IO and Resources II** ---------- 5
- **Test Case 5: CPU Burst with Deadlock** ---------- 6
- **Test Case 6: CPU Burst with Deadlock II** ---------- 8

Table of Figures

Figure I: Test Case One ---------- 2
Figure II: Test Case Two ---------- 3
Figure III: Test Case Three ---------- 4
Figure IV: Test Case Four ---------- 5
Figure VI: Test Case Five ---------- 6
Figure VII: Test Case Six ---------- 8

List of Tables

Table I: Test Case One, Simple CPU Burst without IO ---------- 2
Table II: Test Case Two, CPU Burst with IO and Resource ---------- 3
Table III: Test Case Three, CPU Burst with IO and Resources ---------- 4
Table IV: Test Case Four, CPU Burst with IO and Resources II ---------- 5
Table VI: Test Case Five, CPU Burst with Deadlock ---------- 6
Table VII: Test Case Six, CPU Burst with Deadlock II ---------- 8

Introduction

Deadlock detection and recovery in CPU scheduling are some of the most important concepts in operating systems, which help in managing the system resources effectively. CPU scheduling is responsible for the selection of the sequence of processes to be executed on a CPU, while deadlock detection and recovery ensure that processes never reach a state where they cannot proceed further due to resource contention. The following project simulates a CPU scheduling system that manages processes with both CPU and I/O bursts. Besides, this project will handle resource allocation, requests, and releases. Priority scheduling with round-robin and mechanisms for deadlock detection and resolution have been implemented.

Deadlock is a situation where a set of processes cannot continue because each process in the set waits for at least one resource held by another process in that set, creating a cycle. This situation prevents progress and can lead to system inefficiency. To mitigate this, the project implements a deadlock detection algorithm based on resource allocation graphs, which helps identify cycles that indicate deadlock. Once detected, the system utilizes recovery strategies, such as process termination or resource preemption, to break the deadlock and allow the system to continue functioning.

The simulation begins by reading an input file, which includes a list of process information: PID, arrival time, priority, and a sequence of CPU and I/O bursts with their respective resource requests and releases. The system prioritizes the processes and then schedules them by using a round-robin algorithm, having a fixed quantum of time so that starvation does not occur and fairness is ensured. It also monitors the resource availability and usage so that the resources would be allocated to the needs of the processes concerned and would be released when not required.

The three major objectives of the project are to simulate a priority-based scheduling system with round-robin, to implement a mechanism for deadlock detection, and to apply the recovery methods for deadlock resolution. Also, a Gantt chart shall be prepared which indicates the sequence in which all these processes have been executed, including the actual time for resource utilization. This project includes simulations on CPU scheduling, deadlock detection, and its recovery; thus, making this project significant and real instances of challenges faced by any modern OS in handling its resources by using proper scheduling and deadlock handling mechanisms.

Test Cases

Test Case 1: simple CPU Burst without IO

PID	Arrival Time	Priority	CPU / IO Burst
1	0	3	CPU{4}
2	0	2	CPU{5}
3	0	2	CPU{8}
4	0	1	CPU{7}
5	0	3	CPU{3}

Table I: Test Case One, Simple CPU Burst without IO

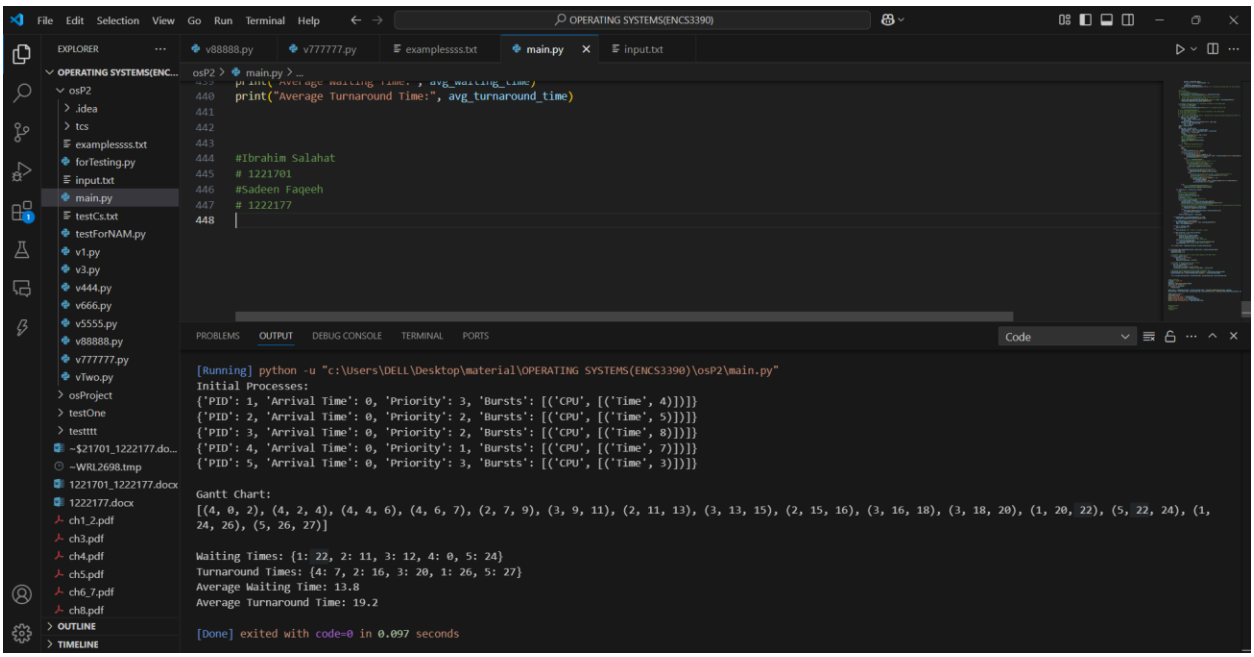


Figure I: Test Case One

- ⇨ So, in this case, we have processes with single CPU burst and without IO.
- ⇨ This case already in slide 27 chapter 5, so the solution is true.

Test Case 2: CPU Burst with IO and Resource

PID	Arrival Time	Priority	CPU / IO Burst
1	0	3	CPU{4}
2	0	2	CPU{5}
3	0	2	CPU{8} IO{3} CPU{3,R[1],4,F[1],2}
4	0	1	CPU{7} IO{3} CPU{3}
5	0	3	CPU{3} IO{3} CPU{3,R[1],4,F[1],2}

Table II: Test Case Two, CPU Burst with IO and Resource

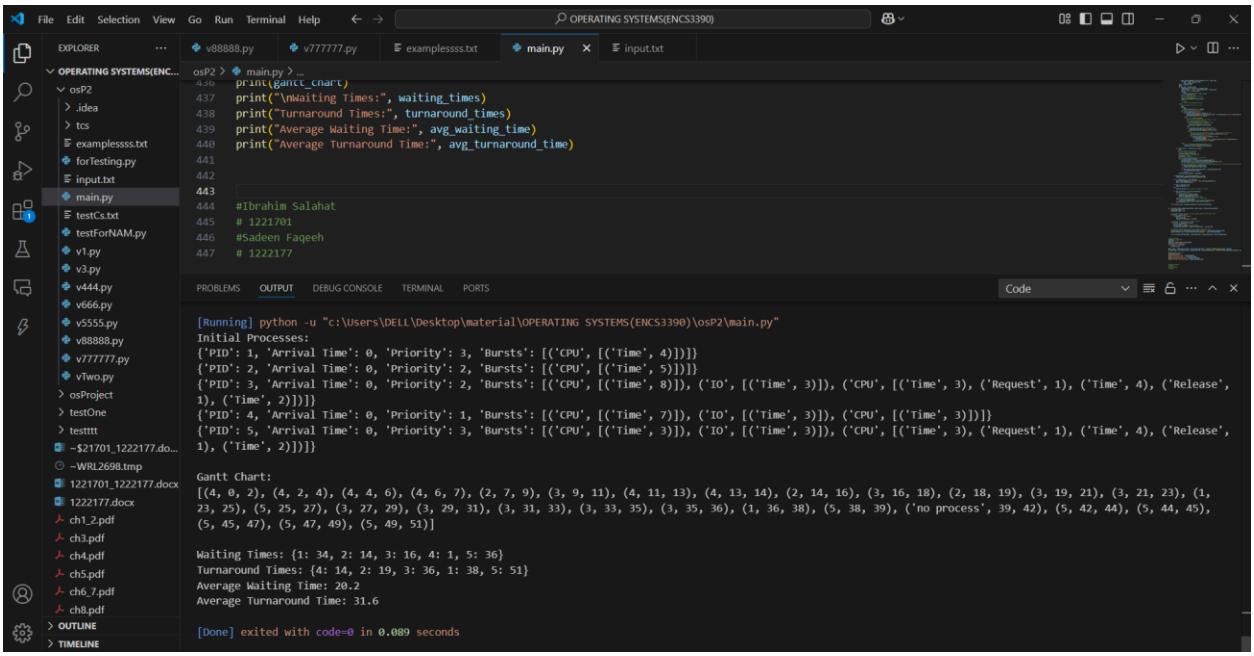


Figure II: Test Case Two

- ⇨ Here, the processes have the same initial arrival times as Case 1 but include additional I/O operations and resource requests in their execution patterns. The scheduler alternates between processes, considering both their priorities and the completion of I/O or resource operations.
- ⇨ Turnaround Time = the time when the process finished it's bursts – arrival time

Test Case 3: CPU Burst with IO and Resources

PID	Arrival Time	Priority	CPU / IO Burst
1	6	3	CPU{4}
2	4	2	CPU{5}
3	0	2	CPU{8} IO{3} CPU{3,R[2],4,F[1],2}
4	0	1	CPU{R[1],7,F[1]} IO{3} CPU{3 }
5	10	3	CPU{3} IO{3} CPU{3,R[3],4,F[1],2}
6	4	0	CPU{R[1],3,F[1]}

Table III: Test Case Three, CPU Burst with IO and Resources

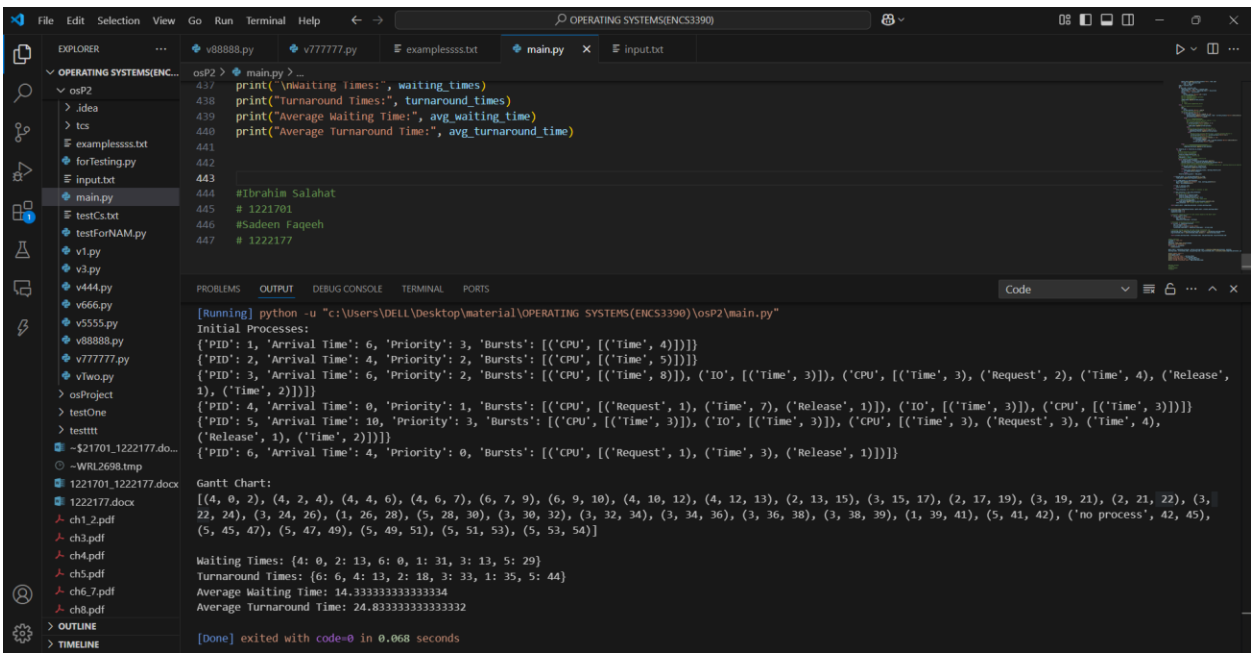


Figure III: Test Case Three

- ⇨ The Gantt chart represents the sequence in which processes are executed, showing the time intervals during which each process runs. Each tuple in the chart follows the format (Process IDوStart Time,End Time).For example, the first entry (4,0,2) indicates that process 4 starts execution at time 0 and continues until time 2. Similarly, (6,7,9) means process 6 runs from time 7 to 9. The presence of ('noprocess',42,45) suggests a period where no process was ready for execution, leading to idle CPU
- ⇨ Note: The algorithm that we followed in the code is to calculate the time that the process takes in the ready queue.

Test Case 4: CPU Burst with IO and Resources II

PID	Arrival Time	Priority	CPU / IO Burst
0	0	2	CPU{R[1],5,F[1]} IO{3} CPU{3}
1	3	1	CPU{R[1],3,F[1],4}
2	6	2	CPU{4} IO{2} CPU{2}

Table IV: Test Case Four, CPU Burst with IO and Resources II

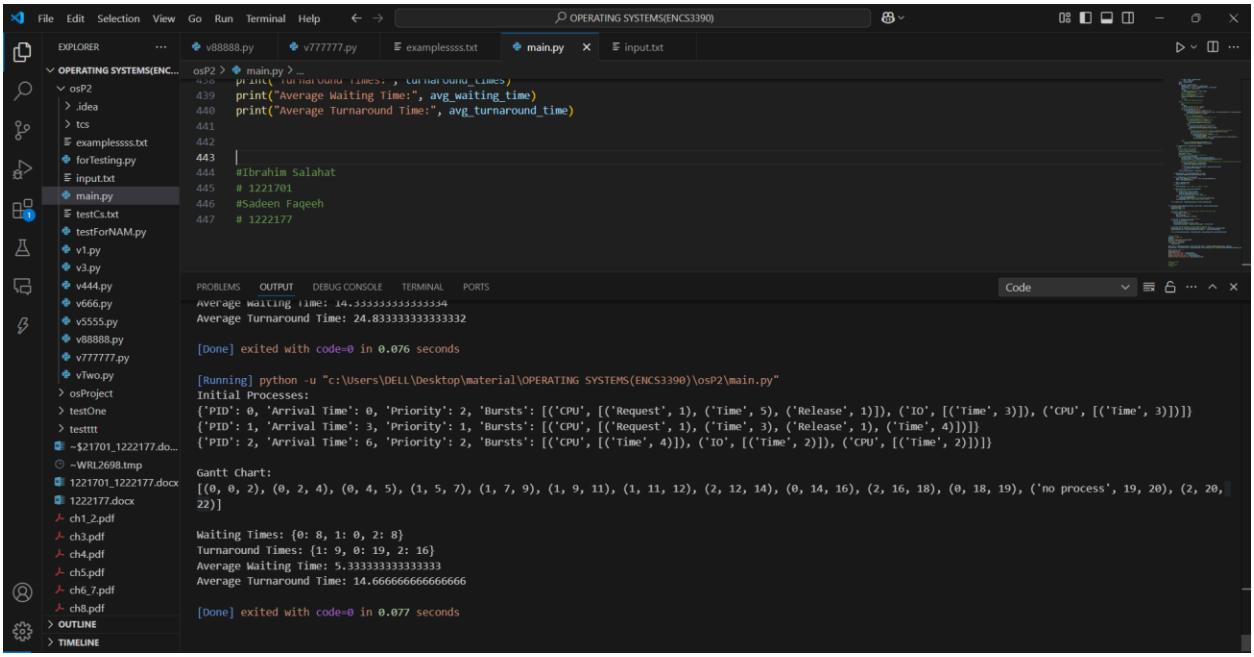


Figure IV: Test Case Four

- ⇨ In this case, processes involve resource requests and I/O operations, with arrival times staggered. Process 0 (P0) begins execution first, followed by Process 1 (P1) and Process 2 (P2), as indicated by the Gantt chart. The scheduler handles resource requests (R[1]) and releases (F[1]), influencing the order of execution.

Test Case 5: CPU Burst with Deadlock

PID	Arrival Time	Priority	CPU / IO Burst
0	0	1	CPU{R[1],15,R[2],10,F[1],F[2]}
1	0	1	CPU{5,R[2],R[1],10,F[2],F[1]}
2	0	0	CPU{50}

Table V: Test Case Five, CPU Burst with Deadlock

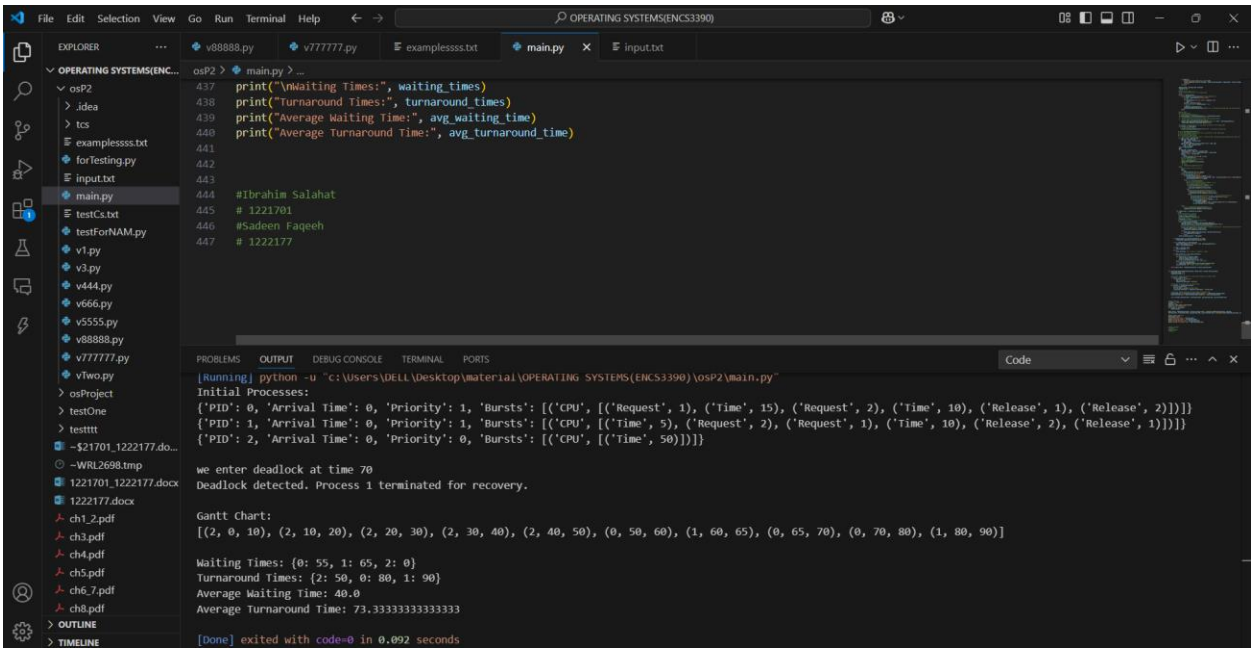


Figure V: Test Case Five

- ⇨ This case highlights a deadlock scenario. Processes 0 (P0) and 1 (P1) request resources that lead to circular waiting, causing a deadlock at time 70. The system detects the deadlock and resolves it by terminating Process 1 (P1). This termination allows Process 0 (P0) and Process 2 (P2) to continue execution, completing their bursts.
- ⇨ Note: During recovery, any interrupted process resumes from where it left off in its burst. The process does not restart its burst from the beginning after recovery. Instead, it continues executing the remaining burst time, ensuring fairness and efficiency in completing the processes.

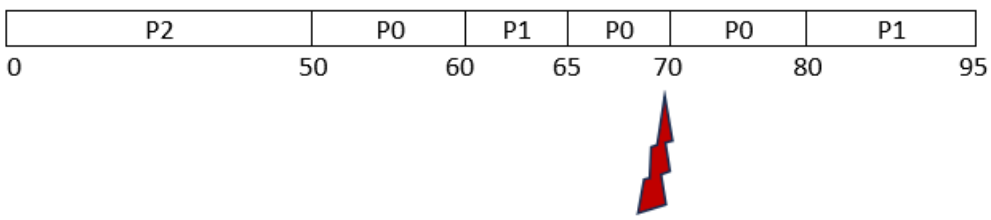
Test case 2 (with deadlock):

Assume the time quantum q = 10, and a lower priority number implies a higher priority

PID	Arrival Time	Priority	CPU / IO Bursts
0	0	1	CPU{R[1], 15, R[2], 10, F[1], F[2]}
1	0	1	CPU[5, R[2], R[1], 10, F[2], F[1]]
2	0	0	CPU{50}

Deadlocked processes: P0 and P1

Gantt Chart (**assuming that deadlock detection is applied whenever a resource is requested and deadlock recovery is achieved by terminating P1, i.e., P1 should run again from the beginning**)



Deadlock is detected, P1 is terminated

and resource 2 is preempted

⇨ Like we can see the output is correct like the one from test cases.

Test Case 6: CPU Burst with Deadlock II

PID	Arrival Time	Priority	CPU / IO Burst
0	0	1	CPU{R[1],3,R[2],2,F[1],F[2]}
1	0	1	CPU{1,R[2],R[1],3,F[2],F[1]}
2	0	2	CPU{3} IO{3} CPU{R[3],2,F[3]}

Table VI: Test Case Six, CPU Burst with Deadlock II

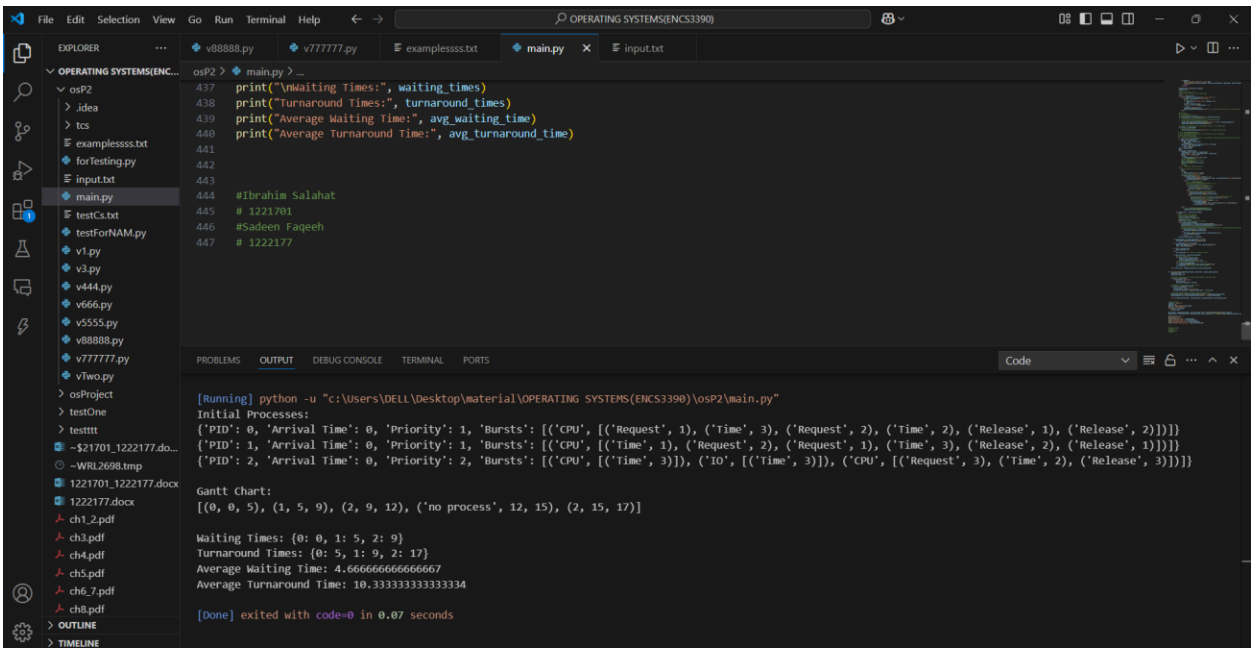


Figure VI: Test Case Six

- ⇨ This case also involves a deadlock, occurring at time 4 due to resource contention between processes. Process 1 (P1) is terminated to recover from the deadlock, allowing the other processes to continue execution. The Gantt chart illustrates how the scheduler alternates between remaining processes to complete their bursts.